Builder Journal

Enterprise Document Intelligence Platform

Maryam Fatima · Track: NLP and AI Agents

Entry dated: July 10, 2026

What worked well?

The core RAG pipeline worked smoothly once everything was connected — uploading a document, chunking it, embedding it, and getting back an answer with the exact source shown. That moment felt like the most rewarding part of the build.

What challenges did you face?

My first real challenge was that my HTML file was not connecting properly to my CSS and JavaScript files. The links I was using were not working as expected. It turned out to be a small mistake, and I was able to fix it myself once I looked closely at the file paths.

My second challenge came up every time I tried to chat with the app — I kept getting an error saying the API was not working. After some digging, I found out the real cause was a required package that had never been installed in my environment. Once I installed it properly, the errors went away and the chat started working as expected.

How did you debug problems?

To debug the API issue, I created a separate test file just to check my Gemini API key on its own, outside of the main Flask app. This let me confirm whether the key and the model itself were working correctly, without the rest of the application getting in the way. Once that test passed, I knew the problem was somewhere else in my code and not with the API key itself, which made it much easier to narrow down where the actual bug was.

What would you improve?

In the beginning, there were quite a few gaps in my UI. There was no way to actually view an uploaded document, no way to delete a chat conversation, and no way to remove an entry from the recent activity list. I went back and improved the interface step by step, taking some design inspiration from Figma AI along the way, and the dashboard looks far more complete now than when I started.

Going forward, I would like to add real user authentication instead of the simple anonymous session I am using now, and continue polishing the UI further. I am still not fully satisfied with how it looks, and I think there is more room to make it feel like a truly professional product.

What did you learn about RAG?

This project really changed how I think about what an AI model can and cannot do on its own. An LLM by itself does not know anything about a company's private documents — it only knows what it was trained on. RAG is the bridge that connects it to that outside knowledge,

instead of leaving it to guess.

A few things clicked for me while building this:

- Embeddings turn text into vectors, and texts with similar meaning end up with similar vectors — that's the whole trick behind searching by meaning instead of exact words.
- A vector database is what makes that meaning-based search (semantic search) actually fast and usable at scale.
- Chunk quality directly decides retrieval quality — and retrieval quality decides how good the final answer is. Bad chunks quietly ruin everything downstream.
- How the prompt is built matters just as much as which model you use. The same context, worded differently, can change the answer.
- Giving the model real retrieved context is what keeps it from hallucinating, and showing the source back to the user is what makes them actually trust the answer.
- RAG is far more practical than fine-tuning when documents keep changing — I don't have to retrain anything, I just re-index the new file.

More than anything, I learned that enterprise AI products are not really about the LLM at all — they're about the retrieval pipeline around it. The model is just the last step in a much longer chain of small decisions.